

# PyDrop – מערכת סנכרון תיקיות מבוססת שרת

Client–Server File Synchronization System

## תיאור כללי

מטרת הפרויקט היא לפתח מערכת לסנכרון קבצים בין מספר מחשבים באמצעות שרת מרכזי. בכל מחשב תפעל תוכנת לקוח שתעקוב אחר תיקייה מקומית. כאשר קובץ נוסף, משתנה או נמחק באחד המחשבים, השינוי יועבר לשרת ולאחר מכן יסונכרן עם יתר המחשבים המחוברים למערכת.

המערכת תמומש בשפת Python ותשתמש בתקשורת TCP באמצעות Sockets. במסגרת הפרויקט יתוכנן פרוטוקול תקשורת עצמאי להעברת הודעות וקבצים בין הלקוחות לשרת.

## ארכיטקטורת הפרויקט

המערכת תתבסס על ארכיטקטורת לקוח-שרת, הכוללת שרת מרכזי ומספר תוכנות לקוח.

### השרת – Server

השרת יהיה אחראי על:

- קבלת חיבורים ממספר לקוחות במקביל.
- החזקת עותק מרכזי ועדכני של התיקייה המסונכרנת.
- שמירת מידע על הקבצים הקיימים במערכת, כגון שם הקובץ, גודלו, זמן העדכון ו-hash של תוכנו.
- קבלת קבצים חדשים או מעודכנים מהלקוחות.
- טיפול בבקשות למחיקת קבצים.
- מסירת עדכונים ללקוחות האחרים.
- שליחת אישור או הודעת שגיאה לאחר כל פעולה.
- טיפול בניתוק ובהתחברות מחדש של לקוחות.

השרת יתמוך במספר לקוחות במקביל, באמצעות Thread נפרד לכל לקוח או מנגנון מקבילי דומה.

### הלקוח – Client

תוכנת הלקוח תפעל על מחשב המשתמש ותהיה אחראית על:

- התחברות לשרת באמצעות TCP Socket.

- מעקב אחר תיקייה מקומית שנבחרה מראש.
- זיהוי הוספה, שינוי או מחיקה של קבצים.
- שליחת השינויים לשרת.
- בדיקה מחזורית אם קיימים עדכונים חדשים בשרת.
- הורדת קבצים מעודכנים ושמירתם בתיקייה המקומית.
- מחיקת קבצים מקומיים שנמחקו בשרת.
- מניעת שליחה חוזרת של שינוי שהתקבל מהשרת.

לצורך הבדיקה ניתן יהיה להריץ מספר לקוחות על אותו מחשב, כאשר כל לקוח יעבוד מול תיקייה מקומית נפרדת.

## מנגנון זיהוי השינויים

כל לקוח יסרוק את התיקייה המקומית במרווחי זמן קבועים, לדוגמה אחת לשתיים או שלוש שניות

בכל סריקה יישמר מידע על מצב התיקייה, הכולל:

- שמות הקבצים.
- גודל כל קובץ.
- זמן השינוי האחרון.
- ערך hash של תוכן הקובץ.

מצב התיקייה החדש ישווה למצב הקודם, וכך ניתן יהיה לזהות:

- קובץ חדש.
- קובץ שתוכנו השתנה.
- קובץ שנמחק.

## מנגנון הסנכרון

כאשר לקוח מזהה שינוי בתיקייה המקומית:

1. הלקוח שולח לשרת הודעה המתארת את סוג השינוי.
2. במקרה של הוספה או שינוי, הלקוח שולח גם את תוכן הקובץ.
3. השרת מעדכן את העותק המרכזי ואת המידע על הקובץ.
4. הלקוחות האחרים בודקים באופן מחזורי אם קיימים עדכונים.
5. כל לקוח מוריד את השינויים ומעדכן את התיקייה המקומית שלו.

## פרוטוקול התקשורת

התקשורת בין הלקוחות לשרת תתבצע באמצעות TCP Sockets.

כל הודעה תכלול כותרת המכילה מידע כגון:

- סוג הפעולה.

- שם הקובץ.
- גודל הקובץ או ההודעה.
- זמן העדכון.
- ערך hash.
- מזהה הלקוח.
- מספר גרסה או מזהה עדכון, לפי הצורך.

סוגי הודעות אפשריים:

- **CONNECT** - התחברות לשרת.
- **LIST\_FILES** - בקשת רשימת הקבצים והגרסאות הקיימות.
- **UPLOAD** - העלאת קובץ חדש.
- **UPDATE** - שליחת גרסה מעודכנת של קובץ.
- **DOWNLOAD** - בקשה להורדת קובץ.
- **DELETE** - הודעה על מחיקת קובץ.
- **CHECK\_UPDATES** - בדיקה אם קיימים עדכונים חדשים.
- **ACK** - אישור שהפעולה הסתיימה בהצלחה.
- **ERROR** - הודעת שגיאה.

הכותרת תישלח בפורמט מובנה, למשל JSON, ולאחריה יישלח תוכן הקובץ במקרה הצורך.

## טיפול בעדכונים מקבילים

במקרה שבו שני לקוחות מעדכנים את אותו קובץ בפרק זמן קצר, השרת יקבע את הגרסה העדכנית לפי סדר קבלת העדכונים.

בגרסה מתקדמת יותר ניתן לשמור עותק נוסף של הקובץ בשם המציין שנוצרה התנגשות, כדי למנוע אובדן מידע.

## גבולות המימוש הראשוני

המימוש הראשוני יתמקד ב:

- שרת מרכזי אחד.
- שני לקוחות לפחות.
- סנכרון אוטומטי של הוספה, שינוי ומחיקה.
- העברת קבצים באמצעות TCP.
- טיפול בקבצי טקסט ובקבצים בינאריים.
- זיהוי שינויים באמצעות סריקה מחזורית.
- טיפול בסיסי בניתוקים ובשגיאות.

הצפנה, הרשאות משתמשים והיסטוריית גרסאות מלאה אינן חלק מדרישות הבסיס של הפרויקט.

# פירוק לתתי-משימות והערכת זמנים

## חלק א' - תכנון הפרויקט ולמידת רקע

### 1. זיהוי הבעיה ובחירת הפתרון

הערכת זמן: 1-2 שעות

- הגדרת מטרת המערכת.
- בחירת ארכיטקטורת לקוח-שרת.
- הגדרת דרישות בסיסיות.
- קביעת גבולות המימוש.
- החלטה על שימוש ב-Python וב-TCP Sockets.

### 2. למידת רקע

הערכת זמן: 3-4 שעות

- לימוד עבודה עם TCP Sockets בפייתון.
- לימוד עקרונות ארכיטקטורת Client-Server.
- קריאה על שליחת קבצים באמצעות Socket.
- לימוד עבודה עם Threads או טיפול במספר לקוחות במקביל.
- לימוד שימוש ב-JSON להעברת metadata.
- לימוד שימוש ב-hashlib לחישוב hash של קבצים.
- קריאה על טיפול נכון בקריאות send ו-recv.

### 3. תכנון הארכיטקטורה

הערכת זמן: 2-3 שעות

- הכנת תרשים של המערכת.
- הגדרת תפקידי השרת והלקוח.
- הגדרת זרימת המידע בין הרכיבים.
- קביעת אופן זיהוי השינויים.
- קביעת אופן הפצת העדכונים.
- הגדרת אופן הטיפול בשגיאות ובניתוקים.

### 4. תכנון פרוטוקול התקשורת

הערכת זמן: 3-4 שעות

- הגדרת סוגי ההודעות.
- הגדרת מבנה כותרת ההודעה.

- קביעת דרך שליחת אורך ההודעה.
- קביעת אופן שליחת קבצים גדולים במספר חלקים.
- הגדרת הודעות אישור ושגיאה.
- תכנון מנגנון למניעת ערבוב בין הודעות שונות.

## 5. כתיבת מסמך העיצוב

הערכת זמן: 3-5 שעות

המסמך יכלול:

- תיאור מטרת המערכת.
- תרשים ארכיטקטורה.
- תיאור רכיבי השרת והלקוח.
- פירוט המחלקות והפונקציות.
- פירוט הפרמטרים וערכי ההחזרה.
- תיאור מבני הנתונים.
- תיאור פרוטוקול התקשורת.
- תיאור תרחישי הפעולה המרכזיים.
- תכנון הבדיקות.

סה"כ חלק התכנון: 12–18 שעות

# חלק ב' – פיתוח התוכנה

## 6. יצירת מבנה הפרויקט

הערכת זמן: 1-2 שעות

- יצירת תיקיות וקבצי קוד.
- הגדרת קובצי קונפיגורציה.
- יצירת תיקיות בדיקה עבור הלקוחות והשרת.
- הגדרת logging בסיסי.
- יצירת קובץ דרישות, אם נעשה שימוש בספריות חיצוניות.

מבנה אפשרי:

None

/PyDrop

```
|
|
|server —|
|
|server.py —|
|
|client_handler.py —|
|
|storage —L|
|
|
|client —|
|
|client.py —|
|
|folder_scanner.py —|
|
|sync_folder —L|
|
|
|common —|
|
|protocol.py —|
|
|file_utils.py —|
|
|constants.py —L|
|
|
|tests —|
|
|test_protocol.py —|
|
|test_file_utils.py —|
|
|test_sync.py —L|
|
```

## 7. מימוש פרוטוקול התקשורת

הערכת זמן: 3-5 שעות

- כתיבת פונקציה לשליחת הודעה.
- כתיבת פונקציה לקבלת מספר בתים מדויק.
- קידוד ופענוח כותרות JSON.
- שליחת אורך הכותרת לפני ההודעה.
- שליחת קבצים במספר chunks.
- החזרת הודעות ACK ו-ERROR.
- טיפול בנתונים חלקיים המתקבלים ב-recv.

## 8. מימוש השרת

הערכת זמן: 5-8 שעות

- פתיחת Socket והאזנה לחיבורים.
- קבלת מספר לקוחות במקביל.
- יצירת Thread או handler לכל לקוח.
- קבלת בקשות העלאה, הורדה ומחיקה.
- שמירת קבצים בתיקיית השרת.
- ניהול metadata על הקבצים.
- החזרת רשימת קבצים וגרסאות.
- טיפול בשגיאות ובניתוק לקוח.
- כתיבת log של פעולות המערכת.

## 9. מימוש סריקת התיקייה המקומית

הערכת זמן: 3-5 שעות

- סריקת הקבצים בתיקייה.
- חישוב hash לכל קובץ.
- שמירת snapshot קודם.
- השוואה בין שני snapshots.
- זיהוי הוספה, שינוי ומחיקה.
- סינון קבצים זמניים או קבצי מערכת.
- מניעת זיהוי שגוי של קובץ שהתקבל מהשרת כשינוי מקומי חדש.

## 10. מימוש תוכנת הלקוח

### הערכת זמן: 5-8 שעות

- התחברות לשרת.
- שליחת מידע ראשוני על מצב התיקיה.
- העלאת קובץ חדש.
- שליחת קובץ מעודכן.
- שליחת הודעת מחיקה.
- בדיקת עדכונים מהשרת.
- הורדת קבצים.
- עדכון התיקיה המקומית.
- טיפול בניתוק ובהתחברות מחדש.

## 11. חיבור כל רכיבי המערכת

### הערכת זמן: 3-5 שעות

- חיבור סורק התיקיה לתוכנת הלקוח.
- חיבור הלקוח לפרוטוקול.
- חיבור השרת למנגנון האחסון.
- בדיקת סנכרון בין שני לקוחות.
- תיקון בעיות של race conditions.
- תיקון סנכרון חוזר או אינסופי.
- טיפול במצב שבו לקוח מתחבר לאחר שהיה מנותק.

### סה"כ פיתוח: 20-33 שעות

---

## חלק ג' – בדיקות

## 12. כתיבת Unit Tests

### הערכת זמן: 3-5 שעות

בדיקות אפשריות:

- חישוב hash לקובץ.
- יצירת snapshot של תיקיה.
- זיהוי קובץ חדש.
- זיהוי שינוי בקובץ.
- זיהוי מחיקת קובץ.
- קידוד ופענוח הודעת פרוטוקול.
- שליחה וקבלה של כותרת.
- בדיקה של שם קובץ לא חוקי.

- בדיקת קובץ ריק.

## 13. בדיקות מערכת ו־Global Tests

הערכת זמן: 4-6 שעות

תרחישי בדיקה:

1. לקוח ראשון מוסיף קובץ והקובץ מופיע אצל הלקוח השני.
2. לקוח משנה תוכן של קובץ והגרסה החדשה מסתנכרנת.
3. לקוח מוחק קובץ והקובץ נמחק אצל שאר הלקוחות.
4. שני לקוחות מתחברים לשרת במקביל.
5. לקוח מתנתק ומתחבר מחדש.
6. לקוח מתחבר כאשר כבר קיימים קבצים בשרת.
7. העברת קובץ ריק.
8. העברת קובץ בינארי, למשל תמונה.
9. העברת קובץ גדול המחייב מספר קריאות **recv**.
10. ניסיון להוריד קובץ שאינו קיים.
11. שליחת הודעה שגויה לשרת.
12. כיבוי השרת בזמן שלקוח מחובר.
13. שני לקוחות מעדכנים את אותו קובץ.
14. בדיקה שלא נוצר לולאת סנכרון אינסופית.

## 14. תיקון תקלות ושיפור יציבות

הערכת זמן: 3-5 שעות

- תיקון תקלות שהתגלו בבדיקות.
- שיפור הודעות השגיאה.
- טיפול בחריגות.
- שיפור ניהול החיבורים.
- בדיקת תקינות הקבצים לאחר ההעברה.
- בדיקת hash לפני ואחרי ההעברה.

סה"כ בדיקות ותיקונים: 10-16 שעות

# חלק ד' – סיום הפרויקט והגשה

## 15. השלמת תיעוד הקוד

הערכת זמן: 2-3 שעות

- הוספת docstrings.
- הוספת הערות במקומות מורכבים.
- תיעוד מחלקות ופונקציות.
- תיעוד מבנה הפרוטוקול.
- שיפור שמות משתנים ופונקציות.
- הסרת קוד זמני וקוד שאינו בשימוש.

## 16. כתיבת מדריך התקנה והפעלה

הערכת זמן: 2-3 שעות

המדריך יכלול:

- דרישות מוקדמות.
- גרסת Python.
- התקנת ספריות.
- הפעלת השרת.
- הפעלת לקוח.
- בחירת תיקייה לסנכרון.
- הגדרת כתובת IP ופורט.
- הרצת מספר לקוחות.
- דוגמה לתרחיש בדיקה.
- פתרון בעיות נפוצות.

## 17. הכנת תיק הפרויקט

הערכת זמן: 2-4 שעות

תיק הפרויקט יכלול:

- מסמך למידת רקע.
- מסמך עיצוב.
- תכנון הבדיקות.
- תוצאות הבדיקות.
- קוד המקור.
- מסמכי התקנה והפעלה.
- צילומי מסך.
- תרשימי המערכת.
- סיכום תהליך העבודה.

## 18. הכנת מצגת

הערכת זמן: 2-4 שעות

המצגת תכלול:

- הבעיה שהפרויקט פותר.
- מטרת הפרויקט.
- ארכיטקטורת המערכת.
- פרוטוקול התקשורת.
- תהליך הסנכרון.
- אתגרים מרכזיים.
- תוצאות הבדיקות.
- הדגמה קצרה.
- סיכום והרחבות אפשריות.

## 19. הכנת הדגמה

### הערכת זמן: 1-2 שעות

- הפעלת השרת.
- הפעלת שני לקוחות.
- הוספת קובץ.
- שינוי קובץ.
- מחיקת קובץ.
- הצגת סנכרון אוטומטי.
- הכנת קבצים מראש למקרה של תקלה בזמן ההצגה.

### סה"כ סיום והגשה: 9-16 שעות

## הערכת זמן כוללת

| שלב               | הערכת זמן  |
|-------------------|------------|
| למידת רקע ותכנון  | 12-18 שעות |
| פיתוח התוכנה      | 20-33 שעות |
| בדיקות ותיקונים   | 10-16 שעות |
| תיעוד, מצגת והגשה | 9-16 שעות  |

הערכת הזמן כוללת עבודה של שני חברי הצוות יחד. ניתן לחלק חלק מהמשימות ביניכם ולעבוד במקביל, ולכן זמן העבודה בפועל לפי לוח שנה עשוי להיות קצר יותר.

## חלוקת עבודה אפשרית בין חברי הצוות

### חבר צוות א'

- מימוש השרת.
- טיפול במספר לקוחות.
- אחסון הקבצים וה-metadata.
- מימוש פעולות העלאה, הורדה ומחיקה.
- כתיבת Unit Tests לשרת.

### חבר צוות ב'

- מימוש הלקוח.
- סריקת התיקייה וזיהוי שינויים.
- הורדת עדכונים ועדכון התיקייה המקומית.
- כתיבת Unit Tests ללקוח ולניהול הקבצים.

### משימות משותפות

- תכנון הפרוטוקול.
- חיבור השרת והלקוח.
- בדיקות אינטגרציה.
- תיקון תקלות.
- כתיבת מסמכים.
- הכנת המצגת וההדגמה.

## תוספות אפשריות לשיפור הפרויקט והציון

התוספות הבאות אינן חלק ממימוש הבסיס. הן יפותחו רק לאחר שהסנכרון המרכזי עובד באופן מלא ויציב.

## 1. ממשק גרפי

פיתוח ממשק באמצעות Tkinter או PyQt, שיציג:

- מצב החיבור לשרת.
- התיקיה המסונכרנת.
- רשימת קבצים.
- פעולות הסנכרון האחרונות.
- הודעות שגיאה.
- כפתור התחברות והתנתקות.

רמת קושי: בינונית.

תרומה אפשרית: משפר מאוד את ההדגמה ואת חוויית המשתמש.

## 2. פס התקדמות להעברת קבצים

הצגת אחוז ההעלאה או ההורדה של קובץ.

רמת קושי: קלה-בינונית.

תרומה אפשרית: תוספת חזותית טובה, במיוחד בקבצים גדולים.

## 3. תמיכה בתיקיות משנה

סנכרון מבנה מלא של תיקיות וקבצים באופן רקורסיבי.

רמת קושי: בינונית.

תרומה אפשרית: הופכת את המערכת לדומה יותר לשירות סנכרון אמיתי.

## 4. טיפול מתקדם בהתנגשויות

במקרה ששני לקוחות משנים את אותו קובץ, המערכת תשמור את שתי הגרסאות:

None

report.txt

report\_conflict\_client\_2.txt

רמת קושי: בינונית.

תרומה אפשרית: מציגה חשיבה על בעיה אמיתית במערכות מבוזרות.

## 5. שמירת היסטוריית גרסאות

השרת ישמור מספר גרסאות קודמות של כל קובץ, ויאפשר שחזור.

**רמת קושי:** בינונית-גבוהה.

**תרומה אפשרית:** מוסיפה עומק משמעותי לפרויקט.

## 6. אימות תקינות באמצעות hash

לאחר הורדת קובץ, הלקוח יחשב hash וישווה אותו ל-hash שנשלח מהשרת.

**רמת קושי:** קלה.

**תרומה אפשרית:** מומלץ מאוד; משפר את אמינות העברת הקבצים.

## 7. התחברות משתמשים

הוספת שם משתמש וסיסמה, כאשר לכל משתמש תיקייה מסונכרנת נפרדת.

**רמת קושי:** בינונית.

**תרומה אפשרית:** הופכת את המערכת למלאה וריאלית יותר.

## 8. הצפנת התקשורת

שימוש ב-SSL/TLS כדי להצפין את המידע המועבר בין הלקוח לשרת.

**רמת קושי:** בינונית-גבוהה.

**תרומה אפשרית:** משמעותית מבחינת אבטחת מידע, אך אינה מומלצת לפני שהמערכת הבסיסית יציבה.

## 9. דחיסת קבצים לפני שליחה

דחיסת קבצים באמצעות **gzip** או **zlib** ושליחתם בצורה דחוסה.

**רמת קושי:** קלה-בינונית.

**תרומה אפשרית:** מאפשרת להציג מדידה של חיסכון בנפח התעבורה.

## 10. מדידת ביצועים

איסוף והצגה של:

- זמן העברת קובץ.
- קצב העברה.
- גודל לפני ואחרי דחיסה.
- מספר הודעות שנשלחו.
- זמן סנכרון ממוצע.

**רמת קושי: קלה.**  
**תרומה אפשרית:** מצוינת לדו"ח ולמצגת, משום שהיא מספקת תוצאות מדידות.

## **סדר עדיפויות מומלץ לתוספות**

לאחר השלמת המימוש הבסיסי, סדר התוספות המומלץ הוא:

1. בדיקת תקינות באמצעות hash.
2. תמיכה בקבצים בינאריים ובקבצים גדולים.
3. logging מפורט.
4. מדידת ביצועים.
5. ממשק גרפי בסיסי.
6. טיפול בהתנגשויות.
7. תמיכה בתיקיות משנה.
8. היסטוריית גרסאות.
9. התחברות משתמשים.
10. הצפנה.

כך גם אם לא תספיקו הכול, הפרויקט הבסיסי יישאר שלם ועומד בדרישות, וכל תוספת שתספיקו תהיה שיפור ולא תלות הכרחית.